tantor

PG BootCamp Russia 2026 Moscow

PGBootCamp.ru

Как детализация статистики замедляет планировщик PostgreSQL и как это исправлять?

Илья Евдокимов, «Тантор Лабс»

План выступления

Проблемные запросы

Посмотрим, как детализированная статистика может замедлять планировщик

Что такое Most Common Values (MCV)

Разберем, как устроена статистика PostgreSQL и какую роль играет MCV

Текущее состояние патчей

Посмотрим, какие решения уже предложены и на каком этапе они находятся.

Обо мне

- Патчи в версии 18, 19
- Доклады на PGDay и PGBootcamp
- Старший разработчик, Tantor Labs
- Магистратура ВМК МГУ

Проблемный запрос

```
SELECT * FROM _InfoRgSL75680 T1 INNER JOIN _InfoRg51339 T2
ON T2._Period = T1._Period
 AND T2._RecorderTRef = T1._RecorderTRef
 AND T2._RecorderRRef = T1._RecorderRRef
 AND T2._Fld51340RRef = T1._Fld51340RRef
 AND T2._Fld51341RRef = T1._Fld51341RRef
 AND T2._Fld59059RRef = T1._Fld59059RRef
WHERE T2._RecorderTRef = '\\000\\000s221'::bytea
 AND T2._RecorderRRef = '\\200\\374\\234\\334q\\313\\331\\000\\021\\360;j'::bytea
 AND T2._Active = TRUE;
```

План запроса

```
Nested Loop (cost=0.45..7.11 rows=1 width=156) (actual time=0.125..0.126 rows=0 loops=1)
  ->  Index Scan using _inforg51339_2 on _inforg51339 (...)
        Index Cond: (...)
        Filter: _active
  ->  Index Scan using _inforgsl75680_1 on _inforgsl75680 (...) (never executed)
        Index Cond: (...)
```

План запроса

```
Nested Loop (cost=0.45..7.11 rows=1 width=156) (actual time=0.125..0.126 rows=0 loops=1)
  ->  Index Scan using _inforg51339_2 on _inforg51339 (...)
        Index Cond: (...)
        Filter: _active
  ->  Index Scan using _inforgsl75680_1 on _inforgsl75680 (...) (never executed)
        Index Cond: (...)
Planning Time: 1343.982 ms
Execution Time: 0.176 ms
(8 rows)
```

default_statistics_target

```
SHOW default_statistics_target;
 default_statistics_target
---------------------------
 10000
(1 row)
```

default_statistics_target = 10 000

```
Nested Loop (cost=0.45..7.11 rows=1 width=156) (actual time=0.125..0.126 rows=0 loops=1)
  ->  Index Scan using _inforg51339_2 on _inforg51339 (...)
        Index Cond: (...)
        Filter: _active
  ->  Index Scan using _inforgsl75680_1 on _inforgsl75680 (...) (never executed)
        Index Cond: (...)
Planning Time: 1343.982 ms
Execution Time: 0.176 ms
(8 rows)
```

default_statistics_target = 100

```
Nested Loop (cost=0.45..7.11 rows=1 width=156) (actual time=0.125..0.126 rows=0 loops=1)
  ->  Index Scan using _inforg51339_2 on _inforg51339 (...)
        Index Cond: (...)
        Filter: _active
  ->  Index Scan using _inforgsl75680_1 on _inforgsl75680 (...) (never executed)
        Index Cond: (...)
Planning Time: 15.982 ms # 1343.982 ms
Execution Time: 0.142 ms
(8 rows)
```

default_statistics_target

default_statistics_target – число, определяющее какое количество строк попадет в выборку статистики при работе команды autovacuum или ANALYZE

Также это число задает:

- Количество наиболее часто встречающихся значений (Most Common Values, MCV) и их частоты
- Количество корзин гистограммы распределения значений столбца

Most Common Values, MCV

```
SELECT most_common_vals, most_common_freqs
FROM pg_stats
WHERE tablename = 'pg_attribute' AND attname = 'atttypmod';
 most_common_vals |                most_common_freqs
------------------+-----------------------------------------------
 {-1,9,36,16}     | {0.9906063,0.0037005409,0.0031312269,0.001423285}
(1 row)
```

Most Common Values, MCV

```
EXPLAIN SELECT * FROM pg_attribute;
                          QUERY PLAN
-----------------------------------------------------------
 Seq Scan on pg_attribute  (cost=0.00..96.13 rows=3513 width=238)
(1 row)

EXPLAIN SELECT * FROM pg_attribute WHERE atttypmod = -1;
                          QUERY PLAN
-------------------------------------------------------------
 Seq Scan on pg_attribute  (cost=0.00..104.91 rows=3480 width=238)
   Filter: (atttypmod = '-1'::integer) -- 3513 * 0.9906063 = 3480
(2 rows)
```

most_common_freqs

Селективность

$$rows = table_size \times selectivity$$

MCV-списки

```
SELECT tablename, attname, array_length(most_common_vals, 1) AS array_length
FROM pg_stats
WHERE tablename IN ('_InfoRgSL75680', '_InfoRg51339');
   tablename    |     attname    | array_length
----------------+----------------+--------------
_InfoRgSL75680 | _RecorderTRef |        10000
  _InfoRg51339 | _RecorderTRef |        10000
_InfoRgSL75680 | _Fld51340RRef |        10000
  _InfoRg51339 | _Fld51340RRef |        10000
 …
```

Flame Graph

Search ic

eqjoin_selectivity

```
# Selectivity of JOIN ON inner.col = outer.col
selectivity = 0.0
for v_inner, freq_inner in MCV_inner:
    for v_outer, freq_outer in MCV_outer: # Nested loop O(N*M)
        if v_outer.is_matched:
            continue;

        if v_inner == v_outer: # detoast
            selectivity += freq_inner * freq_outer;
            v_outer.mark_as_matched();
            break;
```

detoast

```
# detoast MCV_inner, MCV_outer
selectivity = 0.0
for v_inner, freq_inner in MCV_inner:
    for v_outer, freq_outer in MCV_outer: # Nested loop O(N*M)
        if v_outer.is_matched:
            continue;

        if v_inner == v_outer:
            selectivity += freq_inner * freq_outer;
            v_outer.mark_as_matched();
            break;
# free MCV_inner, MCV_outer
```

Результат с detoast

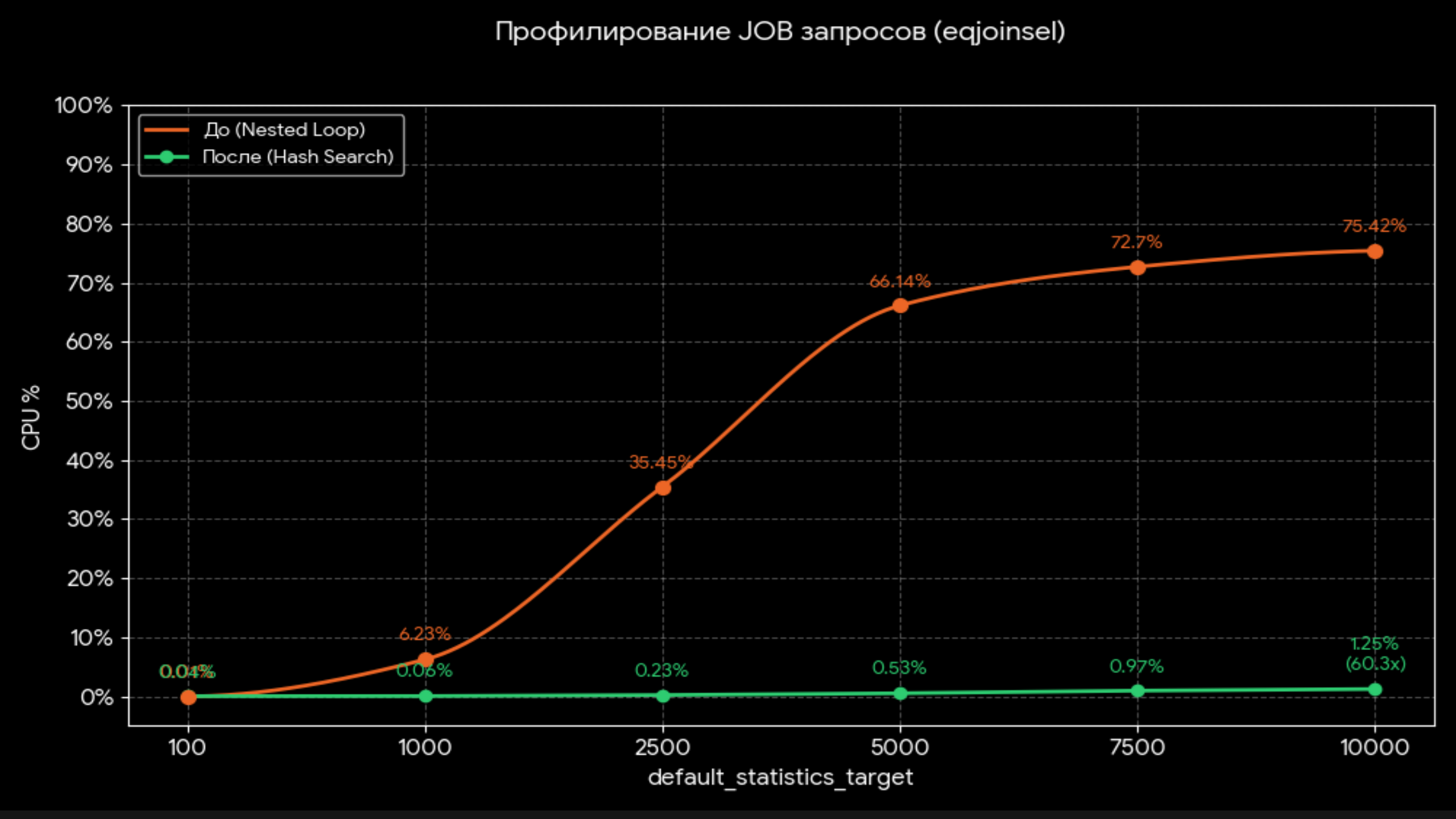
```
Nested Loop (cost=0.45..7.11 rows=1 width=156) (actual time=0.125..0.126 rows=0 loops=1)
  ->  Index Scan using _inforg51339_2 on _inforg51339 (...)
        Index Cond: (...)
        Filter: _active
  ->  Index Scan using _inforgsl75680_1 on _inforgsl75680 (...) (never executed)
        Index Cond: (...)
Planning Time: 723.857 ms # 1343.982 ms
Execution Time: 0.151 ms
(8 rows)
```

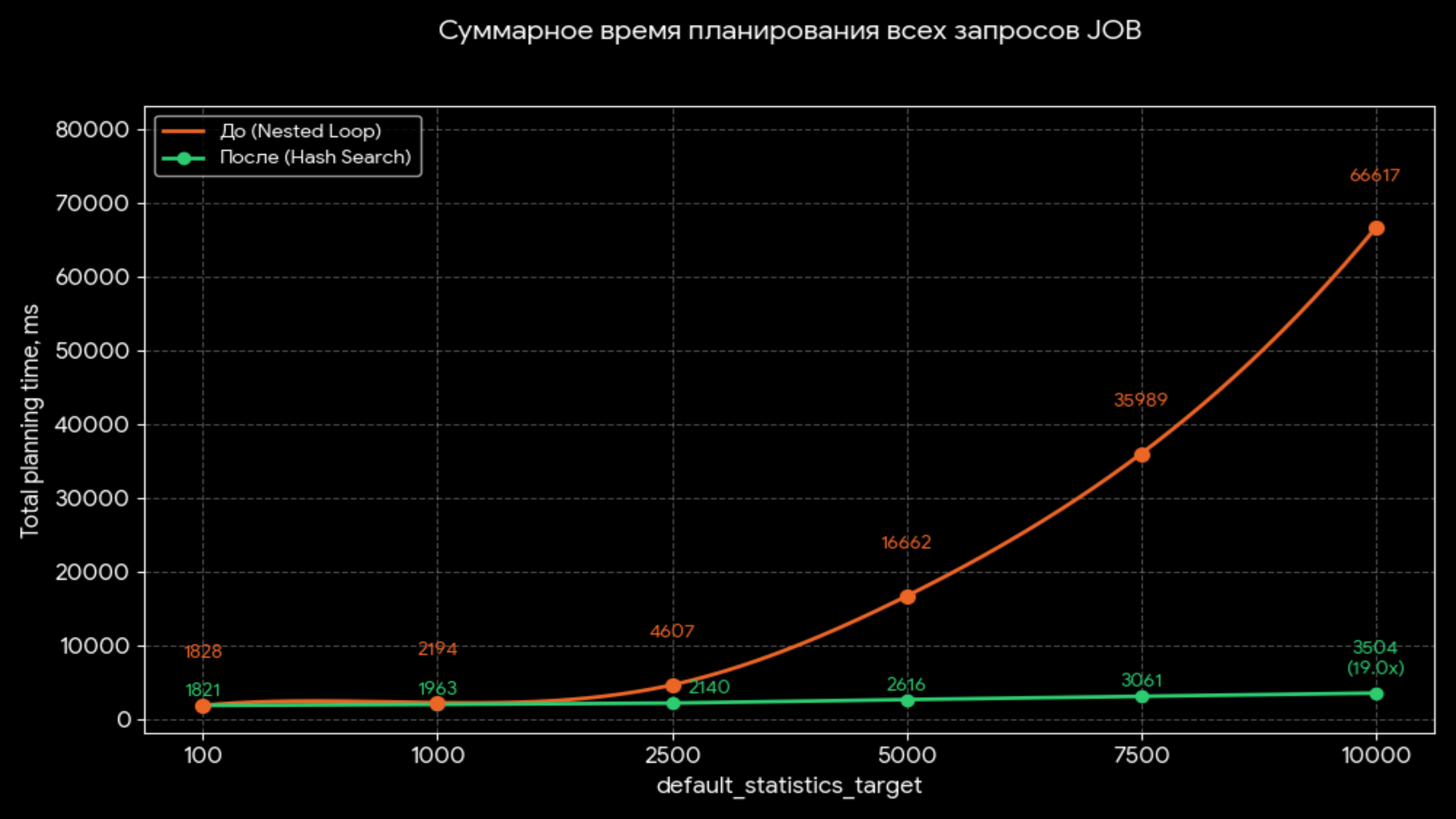
Хэш-поиск

```
selectivity = 0.0

if len(MCV_inner) + len(MCV_outer) >= 200:
    # Hash search -> O(N + M)
    small = min(MCV_inner, MCV_outer, key=len)
    large = max(MCV_inner, MCV_outer, key=len)
    mcv_hash = {v.value: v.freq for v in small}
    for v in large:
        if v.value in mcv_hash:
            selectivity += v.freq * mcv_hash[v.value]
else:
    # Nested loop -> O(N * M)
```

Результат с хэш-поиском

```
Nested Loop (cost=0.45..7.11 rows=1 width=156) (actual time=0.125..0.126 rows=0 loops=1)
  ->  Index Scan using _inforg51339_2 on _inforg51339 (...)
        Index Cond: (...)
        Filter: _active
  ->  Index Scan using _inforgsl75680_1 on _inforgsl75680 (...) (never executed)
        Index Cond: (...)
 Planning Time: 15.857 ms # 1343.982 ms
 Execution Time: 0.077 ms
(8 rows)
```


Профилирование JOB запросов (eqjoinsel)
До (Nested Loop)
После (Hash Search)
CPU %
100%
90%
80%
70%
60%
50%
40%
30%
20%
10%
0%
0.04%
6.23%
35.45%
66.14%
72.7%
75.42%
0.06%
0.23%
0.53%
0.97%
1.25%
(60.3x)
100
1000
2500
5000
7500
10000
default_statistics_target


Суммарное время планирования всех запросов JOB
До (Nested Loop)
После (Hash Search)
Total planning time, ms
80000
70000
60000
50000
40000
30000
20000
10000
0
100
1000
2500
5000
7500
10000
default_statistics_target
1828
2194
4607
16662
35989
66617
1821
1963
2140
2616
3061
3504
(19.0x)

```
commit 057012b205a082ec930cd7bb7f6663516be011e2
Author: Tom Lane <tgl@sss.pgh.pa.us>
Date:   Wed Nov 19 13:22:12 2025 -0500

    Speed up eqjoinsel() with lots of MCV entries.

    If both sides of the operator have most-common-value statistics,
    eqjoinsel wants to check which MCVs have matches on the other side.
    Formerly it did this with a dumb compare-all-the-entries loop,
    which had O(N^2) behavior for long MCV lists.  When that code was
    written, twenty-plus years ago, that seemed tolerable; but nowadays
    people frequently use much larger statistics targets, so that the
    O(N^2) behavior can hurt quite a bit.

    To add insult to injury, when asked for semijoin semantics, the
    entire comparison loop was done over, even though we frequently
    know that it will yield exactly the same results.

    To improve matters, switch to using a hash table to perform the
    matching.  Testing suggests that depending on the data type, we may
    need up to about 100 MCVs on each side to amortize the extra costs
    of setting up the hash table and performing hash-value computations;
    so continue to use the old looping method when there are fewer MCVs
    than that.

    Also, refactor so that we don't repeat the matching work unless
    we really need to, which occurs only in the uncommon case where
    eqjoinsel_semi decides to truncate the set of inner MCVs it
    considers.  The refactoring also got rid of the need to use the
    presented operator's commutator.  Real-world operators that are
    using eqjoinsel should pretty much always have commutators, but
    at the very least this saves a few syscache lookups.

    Author: Ilia Evdokimov <ilya.evdokimov@tantorlabs.com>
    Co-authored-by: David Geier <geidav.pg@gmail.com>
    Reviewed-by: Tom Lane <tgl@sss.pgh.pa.us>
    Discussion: https://postgr.es/m/20ea8bf5-3569-4e46-92ef-ebb2666debf6@tantorlabs.com
```

Scaling PostgreSQL

HOME EPISODES TUTORIALS COURSES ABOUT

16 Times Faster Join Plans | Scaling Postgres 395

Join Over 1,000 Engineers & Get New Episodes Weekly!

TantorLabs 11 дек 2025 в 17:55

Ускорение планирования JOIN'ов — до 16 раз быстрее

Средний 4 мин 13K

Блог компании Тантор Лабс, PostgreSQL*, SQL*, Базы данных*

Обзор Перевод

Автор оригинала: Robins Tharakan

Привет, Хабр! Делимся переводом статьи о патче, сделанном разработчиком «Тантор Лабс» для 19 версии PostgreSQL — по сути, частичкой вклада нашей компании.

Скрытая стоимость избыточной осведомленности. Так можно сказать о ситуации, когда данные распределены неравномерно, целевые значения статистики Postgres установлены высоко, и планировщик пытается оценить стоимость операции JOIN.

Более 20 лет Postgres использовал простой цикл со сложностью $O(N^2)$ для сравнения списков наиболее частых значений (Most Common Values, MCV) во время оценки join. Это работало нормально, когда целевые значения статистики малы (по умолчанию `default_statistics_target` равен 100). Но сегодня опыт PostgreSQL рекомендует увеличивать это значение. Известно, что пользователи используют высокие значения (1000, а иногда и выше), чтобы справиться со сложным распределением данных, и если добавить к этому запрос с 10 соединениями — этот «простой цикл» легко может стать тихим убийцей производительности на этапе планирования.

В **Postgres 19** это меняется.

Проблема: Квадратичная сложность!

+26 55 5

Проблемный запрос ANY

```
CREATE TABLE t (val bytea);
INSERT INTO t SELECT int4send(i) FROM generate_series(1, 10000) AS i,
                                      generate_series(1, 50);
ANALYZE t;
```

```
SELECT * FROM t WHERE val = ANY('{"\\x00000001", "\\x00000002", …,
                                  "\\x00002710"}'); — 10 000 элементов
```

Проблемный запрос ANY

```
EXPLAIN ANALYZE
SELECT * FROM t WHERE val = ANY('{"\\x00000001", "\\x00000002", …,
                                 "\\x00002710"}'); — 10 000 элементов
```

```
Planning Time: 1262.575 ms
Execution Time: 57.293 ms
```

default_statistics_target = 10000

```
EXPLAIN ANALYZE
SELECT * FROM t WHERE val = ANY('{"\\x00000001", "\\x00000002", …,
                                 "\\x00002710"}'); — 10 000 элементов
```

……

```
 Planning Time: 1262.575 ms
 Execution Time: 57.293 ms
```

default_statistics_target = 100

```
EXPLAIN ANALYZE
SELECT * FROM t WHERE val = ANY('{"\\x00000001", "\\x00000002", …,
                                 "\\x00002710"}'); — 10 000 элементов
```

```
Planning Time: 9.018 ms
Execution Time: 56.931 ms
```

ANY-списки

```
selectivity = 0.0;

for any_elem in list:
    s1 = 0.0
    for mcv_value, mcv_freq in MCV_list: # Nested Loop O(N * M)
        if mcv_value == any_elem:
            s1 = mcv_freq;
            break;
    selectivity = selectivity + s1 - (selectivity * s1);
```

Хэш-поиск

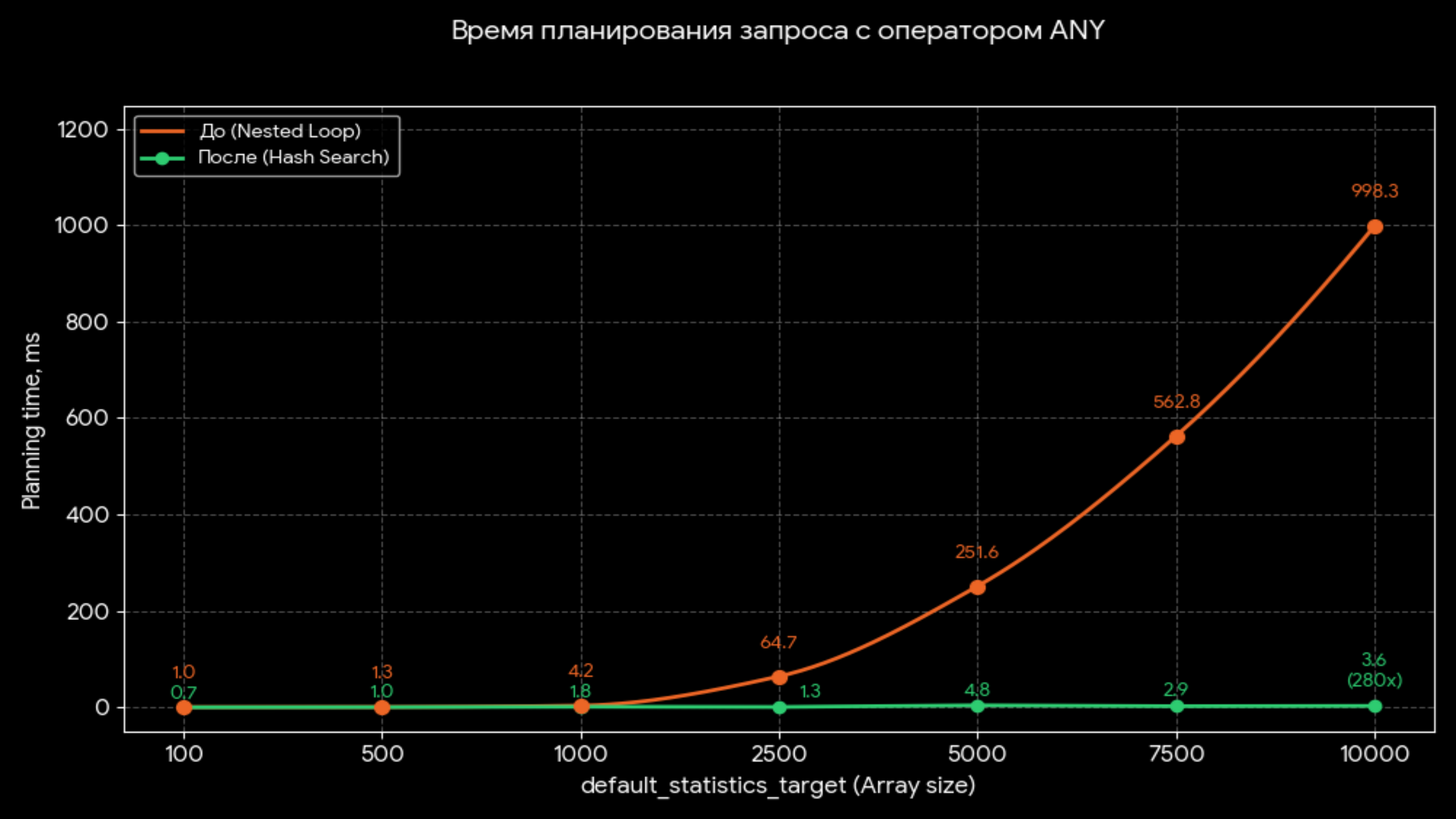
```
selectivity = 0.0

if len(ANY_list) + len(MCV_list) >= 200:
    # Hash search -> O(N + M)
    small = min(ANY_list, MCV_list, key=len)
    large = max(ANY_list, MCV_list, key=len)
    mcv_hash = {v.value: v.freq for v in small}
    for v in large:
        if v.value in mcv_hash:
            sel = mcv_hash[v.value]
            selectivity = selectivity + sel - selectivity * sel
else
    # Nested loop -> O(N * M)
```

default_statistics_target = 10000

```
EXPLAIN ANALYZE
SELECT * FROM t WHERE val = ANY('{"\\x00000001", "\\x00000002", …,
                                 "\\x00002710"}'); — 10 000 элементов
```

……

```
Planning Time: 5.903 ms
Execution Time: 231.636 ms
```


Время планирования запроса с оператором ANY
До (Nested Loop)
После (Hash Search)
Planning time, ms
default_statistics_target (Array size)
0
200
400
600
800
1000
1200
100
500
1000
2500
5000
7500
10000
1.0
0.7
1.3
1.0
4.2
1.8
64.7
1.3
251.6
4.8
562.8
2.9
998.3
3.6
(280x)

Hash-based MCV matching for large IN-lists

Edit | Comment/Review | Change Status

ID	6356
Title	Hash-based MCV matching for large IN-lists
CI (CFBot)	Summary ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ Copy git checkout commands Requeue CFBot
Stats (from CFBot)	Patch version: v6, Patch count: 3, First patch: +94−0, All patches: +686−5
Tags	Performance
Created	2025-12-30 06:53:31
Last modified	2026-02-01 00:00:10 (1 month ago)
Latest email	2026-02-26 08:57:26 (3 days ago)
Status	PG19-Final (2026-03-01 – 2026-03-31): Needs review PG19-4 (2026-01-01 – 2026-01-31): Moved to different CF
Target version	stable
Authors	Ilia Evdokimov (ilidock95)
Reviewers	David Geier (geidav), Tatsuya Kawata (croissant)
Committer	

tantor

PG BootCamp Russia 2026 Moscow

PGBootCamp.ru

tantorlabs.ru

t.me/tantor_labs

Спасибо! Теперь Q&A